

Anamnesis: the architecture we are building toward

A single-container React and Express application in which every forensic claim on screen is traceable to either a measurement performed on the file or a response returned by the model — and nothing else.

ARCHITECTURAL THESIS

One typed contract crosses the network. One pure adapter turns it into what the screen shows. Everything the user sees is downstream of that adapter, and the adapter is forbidden from inventing anything.

Project

Anamnesis Media Forensics Engine

Event

Chandigarh Police National Hackathon 2026

Team

Amity University Punjab

Date

31 August 2026

01 The shape of the system

Anamnesis is one deployable unit: a Node process that serves a React single-page application and, on the same origin, exposes three JSON routes. There is no second service, no message queue, no database. That is a deliberate choice, not a shortcut — every additional moving part is another thing that can fail sixty seconds into a live demonstration.

Three boundaries matter, and only three:

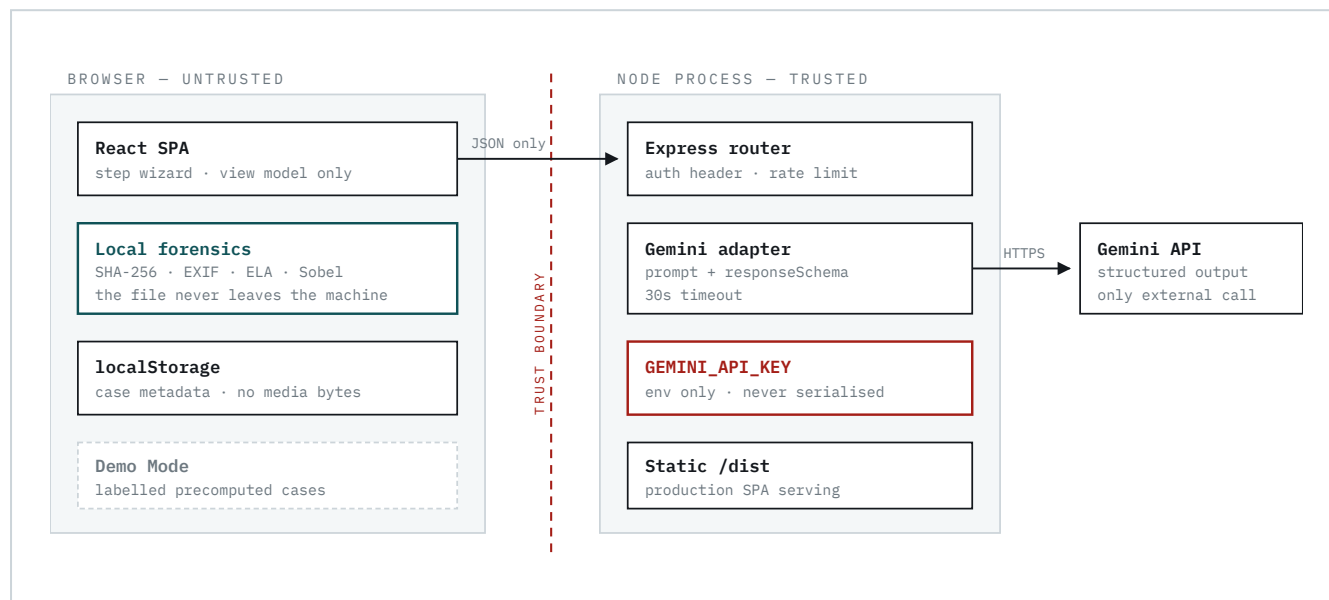


Fig. 1 — Three zones. The API key exists in exactly one of them. The media file, where possible, never leaves the leftmost one.

BOUNDARY	RULE
Browser ↔ Server	JSON over same-origin HTTP. The client never holds a credential and never talks to Gemini directly. The server never trusts a field the client sent without treating it as data.
Server ↔ Gemini	The only outbound network call in the system. Always with a schema, always with a timeout, always logged on failure.
Report ↔ View	The internal boundary that matters most. One function, <code>buildCaseState</code> , crosses it. Nothing else may.

02 The one contract, the one adapter

This is the load-bearing idea of the whole design. Two types and one function between them. If you understand this page you can find your way around every other part of the system.

ANAMNESISFORENSICREPORT

The **wire type**. It is exactly what the Gemini `responseSchema` in `server.ts` produces: case summary, the five questions, the replay timeline, the context-integrity check, technical metrics, origin echo. It is the model's answer, and nothing in the codebase may add fields to an instance of it after it arrives.

PERSISTENTCASESTATE

The **view type**. Organised the way the five-step wizard reads it: `ingest`, `analysis`, `relationships`, `investigation`, `report`. Every step component reads from this and from props — never from the wire type, never from a data file.

```
// src/lib/caseStateBuilder.ts — the single adapter

buildCaseState(intake: MediaIntakeData, report: AnamnesisForensicReport | null)
  → PersistentCaseState

// CONTRACT
// · Pure. Same inputs → same output. No Date.now(), no Math.random().
// · Total. Accepts report === null and returns a valid "not analysed" state.
// · Honest. Every field is copied from `intake` or `report`, or is the
//   explicit NOT_ASSESSED sentinel. It may never originate a fact.
```

THE INVARIANT THAT DEFINES THIS PROJECT

If a value appears in the UI, it came from the file or from the model. No confidence score, platform name, frequency reading, duration, copy count or timestamp may be authored by a developer and rendered as a finding. Where we have no answer, we render `NOT_ASSESSED` — visibly, as a neutral state, never as a green tick.

This is not a style preference. The product's claim is that it assists investigators; a fabricated measurement presented with a confidence percentage is the one failure the audience of this project cannot forgive.

The NOT_ASSESSED state

A first-class value, not an empty string. Every panel that can display a finding must also render a defined "we did not determine this" appearance — grey, unemphasised, with no icon that reads as a verdict. Building this state early is what makes it safe to delete the invented constants, because there is somewhere for the gaps to go.

DISPLAYED AS	WHEN
A finding	The model returned the field, or the browser measured it. Full styling, confidence shown.
NOT_ASSESSED	The model omitted the field, or analysis has not run. Grey, no verdict colour, no percentage.
An error	The call failed. Visibly distinct from both of the above, with a retry affordance.
Precomputed	Demo Mode. Badged in the UI as a reference case, so it is never mistaken for a live result.

03 The ingest flow, end to end

This is the primary path and the one the demonstration follows. Read it as the definition of done for Phases 1 and 3.

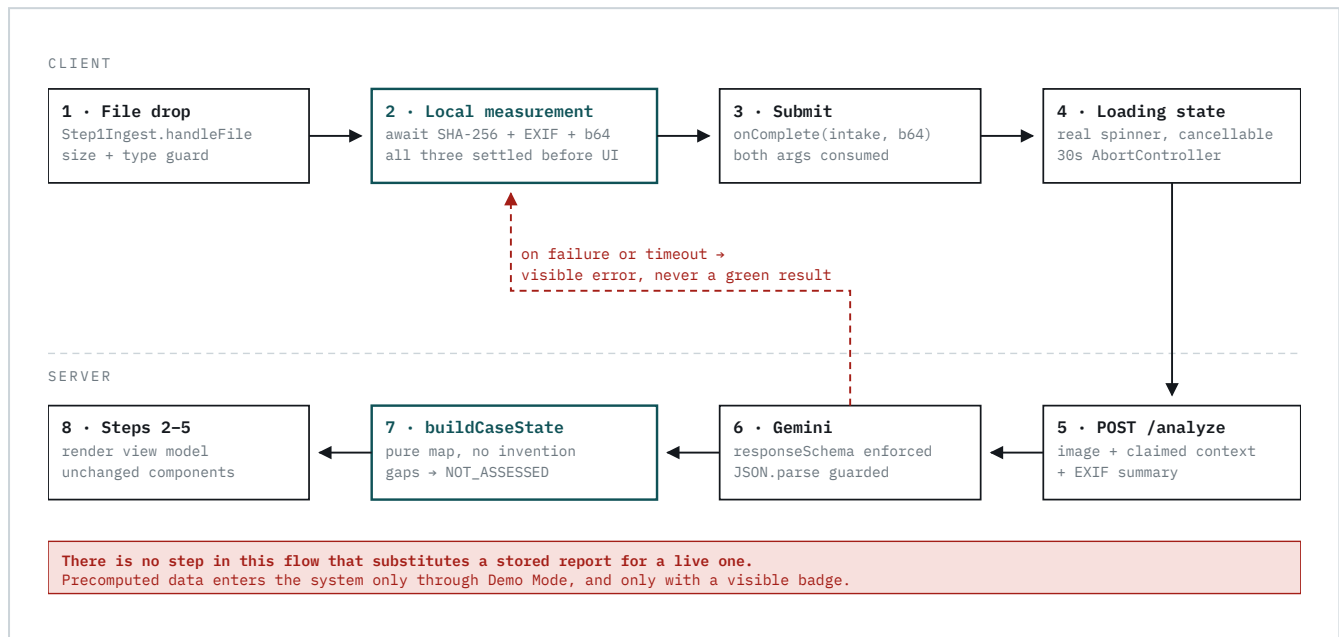


Fig. 2 – The ingest path. Step 7 is the adapter from Fig. 1's third boundary; steps 1–2 are the client-side forensics that need no network at all.

Where this flow is allowed to fail, and what happens

FAILURE	BEHAVIOUR
File too large / wrong type	Rejected at step 1 with a message naming the limit. Never reaches hashing.
Network unreachable	Step 4 surfaces an error card with Retry. Steps 2–5 remain locked. No partial case is written.
Timeout (30s)	Abort fires; same error card. The spinner cannot outlive the request.
Model returns malformed JSON	Server returns a generic 500; raw output goes to server logs only, never to the client.
Model omits optional fields	Not a failure. Adapter maps them to <code>NOT_ASSESSED</code> and the case proceeds.
localStorage quota exceeded	Caught and shown as a visible warning. Cannot happen silently, because media bytes are no longer persisted.

04 Layer responsibilities

Each row is a rule about where a kind of logic is allowed to live. Disputes about "where should this go" are settled here.

LAYER	OWNS	MUST NEVER
Step components <code>src/components/Step*.tsx</code>	Rendering <code>PersistentCaseState</code> . Local interaction state (which tab, which stage). Emitting completion upward.	Fetch. Import from <code>data/</code> . Contain a domain constant.
App shell <code>src/App.tsx</code>	View switching, the step machine, case persistence, all network orchestration for ingest.	Hold a fallback report. Format anything for display.
Adapter <code>src/lib/caseStateBuilder.ts</code>	Wire type → view type. The <code>NOT_ASSESSED</code> decision for every field.	Originate a fact. Call anything impure.
Local forensics <code>src/lib/{cryptoUtils,imageForensics}.ts</code>	SHA-256, EXIF parsing, ELA, noise variance, Sobel. Real measurement on real bytes.	Know about cases, steps or the server.
Presentation utilities <code>src/lib/{statusStyles,soundFx}.ts</code>	The single status→style mapping. Audio feedback.	Be duplicated inside a component.
Reference data <code>src/data/benchmarkCases.ts</code>	Demo Mode fixtures only, each explicitly flagged <code>isPrecomputed: true</code> .	Be reachable from the live ingest path.
Server <code>server.ts</code>	Credential custody, prompt construction, schema enforcement, timeouts, rate limiting, static serving.	Return raw model output or SDK error text to the client.

Why the heavy forensics stays on the client

Hashing, EXIF extraction and Error Level Analysis all run in the browser. This is not a performance decision — it is the product's strongest claim. An investigator can compute a cryptographic digest and inspect compression artefacts on a sensitive file **without that file being transmitted anywhere**. Say this out loud during the demonstration; it is a genuine architectural virtue and it distinguishes the tool from an upload-and-hope service.

Only when the investigator asks for AI interpretation does an image cross the boundary, and that is an explicit, visible action.

05 Directory structure

```
anamnesis/
├─ server.ts           Express + Gemini adapter + static serving
├─ Dockerfile         - NEW: stop depending on the AI Studio wrapper
├─ vite.config.ts
├─ .env.example       GEMINI_API_KEY · GEMINI_MODEL · PORT · API_SHARED_SECRET
└─ src/
  ├─ main.tsx
  ├─ App.tsx          view switch · step machine · ingest orchestration
  ├─ types.ts         AnamnesisForensicReport · PersistentCaseState · NOT_ASSESSED
  ├─ lib/
  │   ├─ caseStateBuilder.ts  THE adapter - pure, total, honest
  │   ├─ api.ts              - NEW: typed fetch wrappers, timeout, error normalisation
  │   ├─ cryptoUtils.ts      SHA-256 + EXIF (unchanged)
  │   ├─ imageForensics.ts   ELA · noise · Sobel (unchanged)
  │   ├─ statusStyles.ts     - NEW: the one getStatusBadge
  │   └─ soundFx.ts          (unchanged)
  ├─ data/
  │   ├─ benchmarkCases.ts   Demo Mode fixtures, flagged isPrecomputed
  │   └─ mockInvestigator.ts prototype identity + storage helpers
  └─ components/
      ├─ Step1Ingest.tsx ... Step5Report.tsx
      ├─ ForensicCanvasLab.tsx → MOUNTED into Step 2
      ├─ ForensicCopilotChat.tsx → MOUNTED into Step 4
      ├─ DossierExportModal.tsx · Navbar · LoginScreen · ...
      └─ (the eight orphaned scene components: deleted)
```

STATUS	MEANING
UNCHANGED	Correct today. Do not refactor it during hackathon preparation; there is no score in it.
MODIFIED	App.tsx , caseStateBuilder.ts , types.ts , server.ts , Step1Ingest , Step2Analyse . Behaviour changes; shape does not.
NEW	Three files: lib/api.ts , lib/statusStyles.ts , Dockerfile . That is the entire net addition.

NOTE ON SCALE

The target architecture adds three files and removes roughly 2,400 lines. It is the same architecture the project already has, with the invented content drained out and the real capability reconnected. Nobody should be moving directories.

06 API surface

Three routes. Both POST routes require the shared-secret header and are rate limited by IP.

ROUTE	IN	OUT
GET /api/health	—	<code>{ status, engine, model, timestamp }</code> — now reports the resolved model ID so a bad configuration is visible before the demo, not during it.
POST /api/forensics/analyze	<code>imageBase64</code> , <code>contentType</code> , <code>fileName</code> , <code>fileHash</code> , <code>exifData</code> , and the claimed context (<code>claimedLocation</code> , <code>claimedDateTime</code> , <code>claimedNarrative</code> , <code>sourcePlatform</code>)	<code>AnamnesisForensicReport</code> , schema-enforced. On error, a generic <code>{ error }</code> with no model output.
POST /api/forensics/chat	<code>message</code> , <code>reportContext</code> , optional <code>imageBase64</code> + <code>contentType</code>	<code>{ reply }</code> . Errors return non-2xx and the client must render them as errors.

TWO CONTRACT RULES THAT ARE EASY TO GET WRONG

Always send `contentType` with image bytes. The server must not guess. An SVG preview sent as `image/jpeg` is rejected by the model, and if the client is not checking `res.ok` that rejection becomes a fake all-clear.

Never render a response body without checking `res.ok` first. Every `fetch` goes through `lib/api.ts`, which throws on non-2xx, so no component can make this mistake individually.

The model configuration

The model ID lives in `process.env.GEMINI_MODEL` with a verified default, is logged at process start, and is echoed by `/api/health`. It appears as a literal in zero places. This exists so that "is the model right" is answerable in one `curl` rather than by watching a demo fail.

07 State, persistence and identity

Client state

All application state lives in `App.tsx` and flows down as props. No context provider, no store library. The tree is two levels deep and the state is genuinely shared across the step ladder, so a store would be ceremony. Revisit this only if a third view needs the same state — it does not.

Persistence

PERSISTED	NOT PERSISTED
Case identifier, title, step position, completed steps, the full <code>PersistentCaseState</code> minus media , and the investigator ID it belongs to.	<code>previewUrl</code> , <code>mediaUrl</code> , and the base64 payload. These live in memory as an <code>ObjectURL</code> for the session and are stripped before <code>JSON.stringify</code> .

The storage write is wrapped and its failure is **surfaced**, not logged. Silent persistence loss during a demonstration is worse than an honest warning.

Identity

DELIBERATE SCOPE DECISION

Authentication remains a prototype stub, and we say so — in the file headers, in the README's Future Scope, and if asked, in the room. Real departmental identity, RBAC, encrypted evidence storage and audit logging are the correct production answer and are explicitly out of scope for the hackathon build.

Building real auth costs a day and changes no judging outcome. Being straightforward that it is a stub costs nothing and reads as engineering maturity.

08 AI integration strategy

Two calls, both server-side, both schema-bound where a schema applies.

CALL	WHEN	COST CONTROL
Analyse	Once per ingest, triggered by completing Step 1.	Result cached into case state. Re-rendering, navigating back, or reopening a case never re-bills.
Cross-examine	Once per investigator question, in Step 4.	User-initiated only. No polling, no background refresh, no auto-send on mount.

Why structured output rather than free text

The `responseSchema` already in `server.ts` mirrors the five-question methodology field for field. Keeping it is what makes the adapter possible: the client receives a shape it can map mechanically, rather than prose it would have to parse. This is the single best decision in the existing codebase and the target architecture preserves it unchanged.

Prompt hygiene

- Investigator-supplied text (claimed location, narrative, notes) is **data inside the prompt**, never instruction. It is fenced and labelled as claimed context.
- `reportContext` on the chat route is client-supplied and therefore untrusted; it is embedded as reference material, and the system instruction that governs behaviour is server-authored and not client-influenced.
- Every response is presented as an assisted finding requiring human verification — which the footer already states and the report must continue to state.

DEGRADATION POLICY

If the AI is unreachable, the product must still be useful: SHA-256, EXIF and ELA all run locally and require no network. A failed analyse call yields a case with real local measurements and `NOT_ASSESSED` for everything interpretive. That is a legitimate, demonstrable state — and it is the reason Phase 3 mounts the canvas lab.

09 Deployment

One container, one port, one environment file.

```
# Runtime configuration – the complete list
GEMINI_API_KEY      required · server only · never serialised to a response
GEMINI_MODEL        defaults to a verified ID · logged at startup
API_SHARED_SECRET   required by both POST routes
PORT                from the platform; falls back to 3000
NODE_ENV            'development' enables Vite middleware; production is the default
```

DECISION	RATIONALE
Production is the default branch	The current check is <code>!== 'production'</code> , so an unset variable boots a dev server in production. Inverting it to <code>=== 'development'</code> makes the safe path the one that happens by accident.
PORT from environment	Cloud Run injects it. A hardcoded 3000 works only by luck.
Explicit Dockerfile	Four lines that remove a dependency on the AI Studio wrapper's implicit behaviour. Also makes the deployment reproducible on a laptop if the hosted URL fails on the day.
Single artefact	<code>vite build</code> emits <code>dist/</code> ; <code>esbuild</code> emits <code>dist/server.cjs</code> ; <code>node dist/server.cjs</code> serves both. Nothing else to orchestrate.

Demo resilience

- **Run it locally as well as hosted.** The presentation should not depend on conference wifi reaching Cloud Run.
- **Demo Mode is the safety net.** Precomputed reference cases render with zero network calls, so there is always something to show — labelled honestly as precomputed.
- **Rehearse the off-script path.** Have someone hand you an unfamiliar photograph and run the full five steps on it. Every serious defect found in this project would have surfaced the first time anyone did that.

10 What we are deliberately not building

An architecture is defined as much by its exclusions. Each of these is a reasonable idea that would cost more than it returns before the hackathon.

EXCLUDED	REASONING
A database	<code>localStorage</code> is correct for a single-investigator prototype. A database is a deployment dependency that can fail live — precisely the risk this architecture is built to remove.
Real authentication	Out of scope and honestly labelled. See §07.
A router	The wizard is linear and nobody deep-links to Step 3 during judging. Accept that refresh returns to the start; do not engineer around it.
A state library	Two levels of prop drilling over genuinely shared state. Adding Redux or Zustand would be ceremony.
A second service	No queue, no worker, no separate API host. Analysis is a single request-response.
Splitting <code>App.tsx</code>	855 lines, but overwhelmingly declarative JSX for the step ladder rather than tangled logic. Splitting risks the render tree for no visible gain.
Bundle optimisation	809 KB loads fine on any connection a demonstration will use. Code-splitting jsPDF is a post-hackathon nicety.
A test suite	With a fixed date and a demo that will be manually rehearsed anyway, rehearsal is the higher-value verification. Write tests afterwards.

THE ONE THING THAT IS NOT NEGOTIABLE

Everything in the table above can be revisited, deferred or argued about. The invariant in §02 cannot. A forensic tool that presents an authored constant as a measurement is not an incomplete product — it is a wrong one. Every architectural decision in this document exists to make that failure structurally difficult rather than merely discouraged.

Definition of done

1. `/api/health` reports a model ID that resolves against the live API.
2. Dropping an unfamiliar photograph produces either a real analysis or a visible error — never a stored report.
3. Two different cases opened side by side show different findings.
4. Step 2 renders an ELA computed from the ingested image in the browser.
5. Disconnecting the network mid-flow produces an error card, not a green tick.
6. No file under `src/` is unreachable from `main.tsx`.